



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

FASTAPI PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 Q&A on type hints, Pydantic, async, DI, and OpenAPI

TOTAL: 10 QUESTIONS

Q1 How do Python type hints drive FastAPI's validation and docs?

Threat Model

Q6 How do you implement OAuth2 password flow with JWT in FastAPI?

Observability

Q2 What is a Pydantic model and how does it validate data?

Architecture

Q7 How does FastAPI auto-generate OpenAPI documentation?

Lifecycle

Q3 How does FastAPI's dependency injection work with Depends()?

Configuration

Q8 How do you add CORS support to a FastAPI application?

Compliance

Q4 How does FastAPI handle path operation ordering to avoid shadowing?

Hardening

Q9 How do you use SQLAlchemy async sessions with FastAPI?

Testing

Q5 What is the difference between async def and def in a FastAPI route?

SSO / IdP

Q10 How do you test FastAPI endpoints with TestClient?

Tuning



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 FastAPI reads function parameter annotations at

Mitigation

FastAPI reads function parameter annotations at import time and builds JSON Schema from them using Pydantic. It validates incoming requests against the schema, coerces types, and populates the OpenAPI spec all from the same source of truth.

A6 Use OAuth2PasswordBearer to extract tokens

Observability

Use OAuth2PasswordBearer to extract tokens from Authorization headers. On login, verify credentials, create a JWT with python-jose encoding claims and expiry, and return it. Protect routes with Depends(get_current_user) which decodes and validates the token.

A2 A Pydantic BaseModel subclass defines fields

Primitives

A Pydantic BaseModel subclass defines fields with Python type annotations. On instantiation Pydantic validates each field, coerces compatible types (e.g., '3' to int), and raises ValidationError with detailed messages if validation fails.

A7 FastAPI builds an OpenAPI 3 schema

Automation

FastAPI builds an OpenAPI 3 schema from route decorators, type annotations, and Pydantic models at startup. It serves interactive Swagger UI at /docs and ReDoc at /redoc. Use response_model, tags, and summary arguments to customize the spec.

A3 A dependency is any callable annotated

Hardening

A dependency is any callable annotated in a parameter as Depends(get_db). FastAPI calls it before the route handler, resolves nested Depends(), and injects the result. yield-based dependencies act as context managers for setup/teardown (e.g., DB sessions).

A8 Add CORSMiddleware:

Governance

Add CORSMiddleware: app.add_middleware(CORSMiddleware, allow_origins=['https://myfrontend.com'], allow_methods=['*'], allow_headers=['*']). FastAPI handles preflight OPTIONS requests automatically. Use allow_origins=['*'] only in development.

A4 FastAPI registers routes in declaration order.

Gotchas

FastAPI registers routes in declaration order. A fixed path /users/me must be declared before the parameterized /users/{user_id}; otherwise the router matches 'me' as a user_id value. Declare more specific routes first.

A9 Use create_async_engine with an async driver

Assessment

Use create_async_engine with an async driver (asyncpg for PostgreSQL). Define a get_db dependency that yields an AsyncSession. Annotate all repository methods with async and use await session.execute(stmt). Return results with scalars().all().

A5 async def routes run on the

Federation

async def routes run on the event loop—never block them with synchronous I/O. def routes are automatically run in a threadpool by FastAPI so they don't block the event loop. Use async def with await for async libraries; def for synchronous blocking code.

A10 Instantiate TestClient(app). Call

Optimization

Instantiate TestClient(app). Call client.get('/path') or client.post('/path', json={...}) synchronously. Override dependencies for tests using app.dependency_overrides[get_db] = override_get_db. Use pytest fixtures to manage setup and teardown.